“蜀道山”高校联合公益赛 Writeup

战队信息

战队名称：纵你阅题何其多，再无一题恰似我

战队排名：15

Misc

Elemental Wars

比较抽象的题目，随机打出12345的元素，就有概率获胜，瞎打一通获得了flag。

神奇的硬币纺纱机

打开后获取题目地址

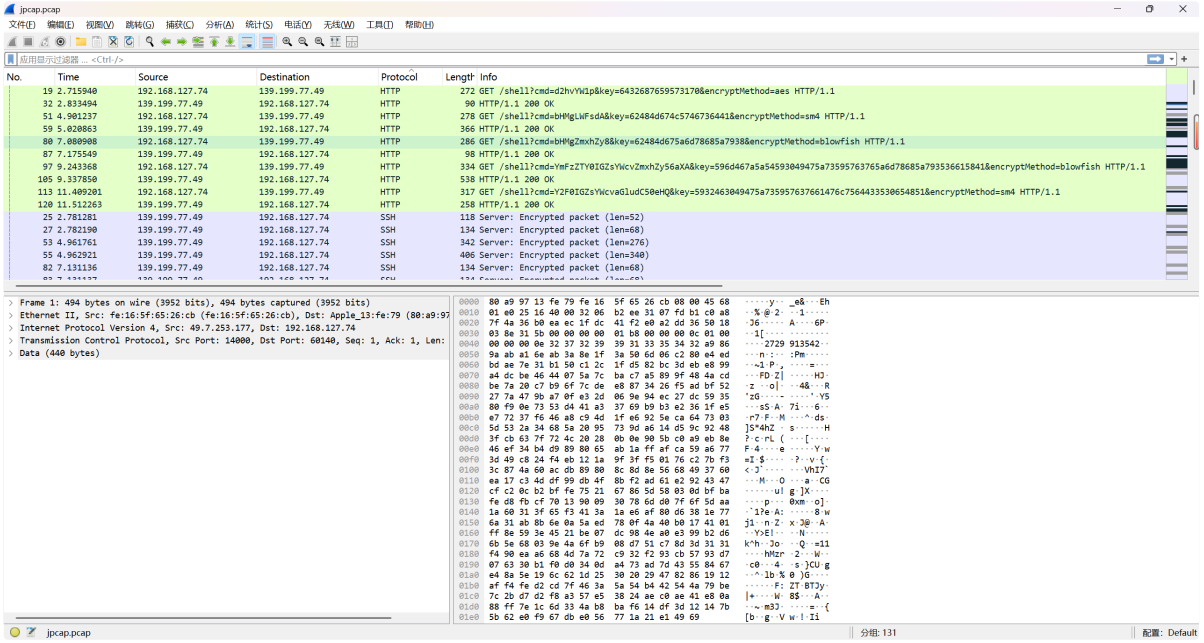
在linux虚拟机内打开终端，用nc连接

多次输入0，偶尔输入1，即可胜利，

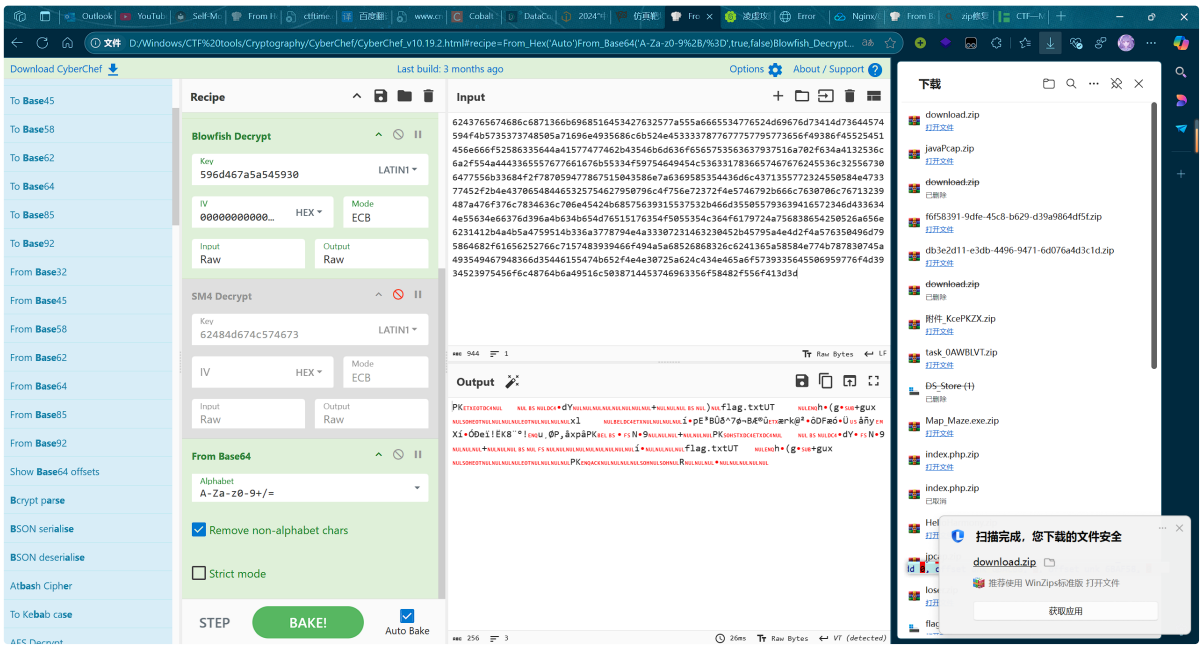
获得flag

javaPcap

wireshark打开，容易看见一些可疑流量包：



结合反编译的jar包，可以解密这个webshell传输的内容，其中密码位数为16比特，使用cyberchef一个个解密，可以找到一个压缩文件和一个提示：

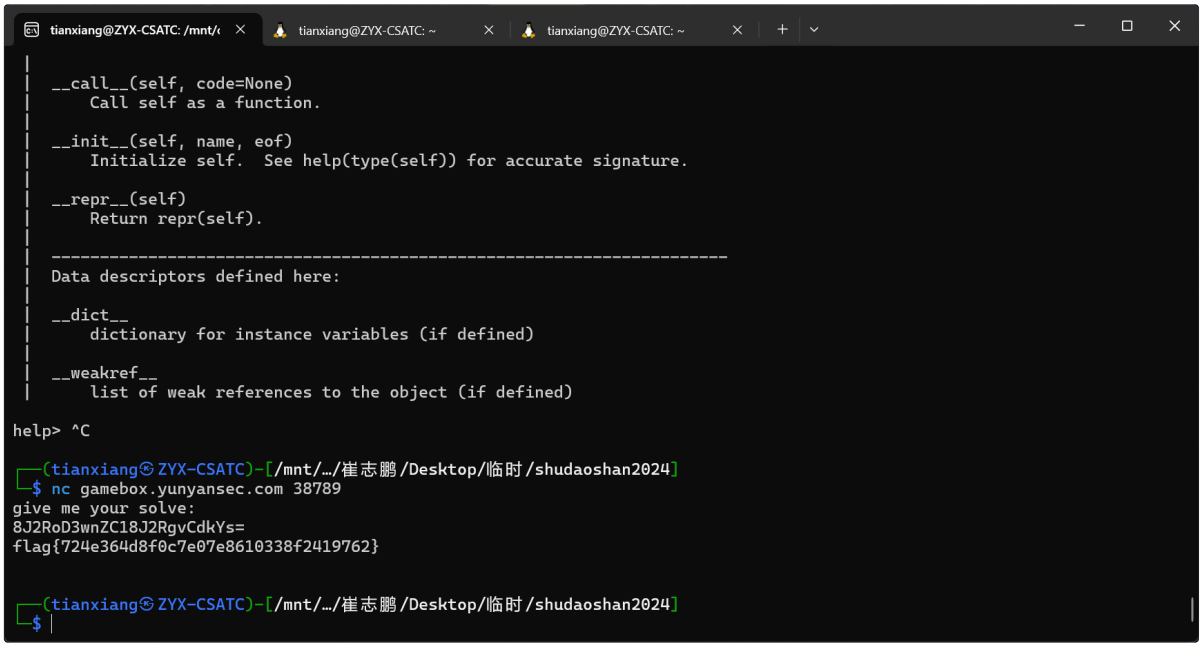


密码为执行命令（按照时间排序）的首字母的组合重复三次，比如执行了（id,whoami），那么密码就为iwiwiw

根据提示可以解压出flag，密码为 `wllbcwllbcwllbc`

golf

屏蔽了许多关键词，使用unicode编码绕过，输入base64编码后的 `s=BOX` 即可获得flag：

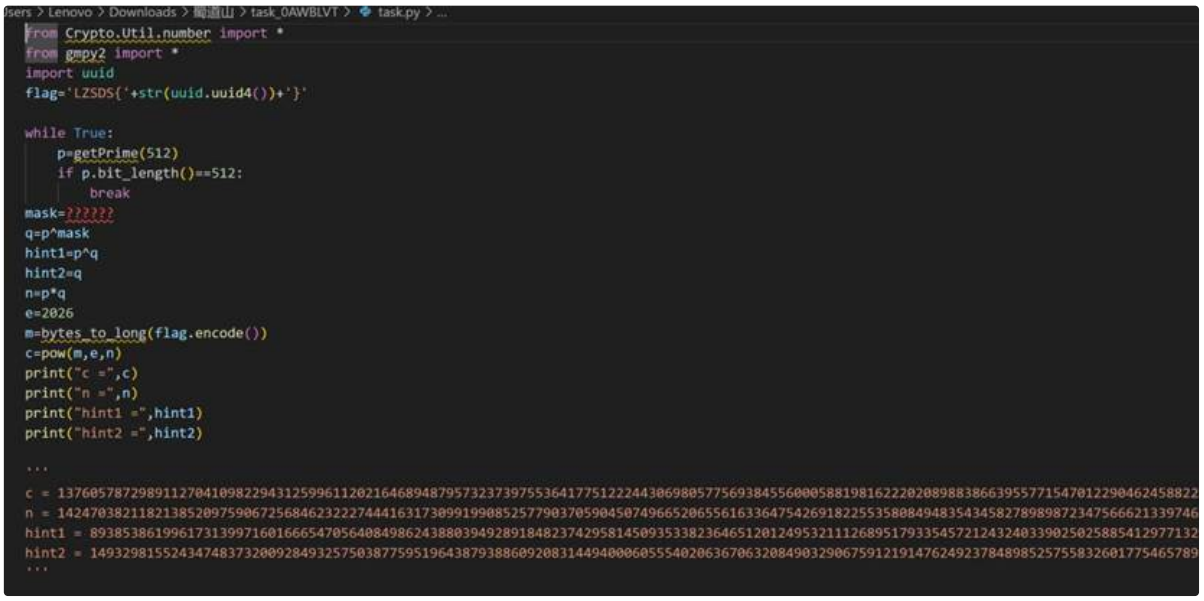


Crypto

xorsa

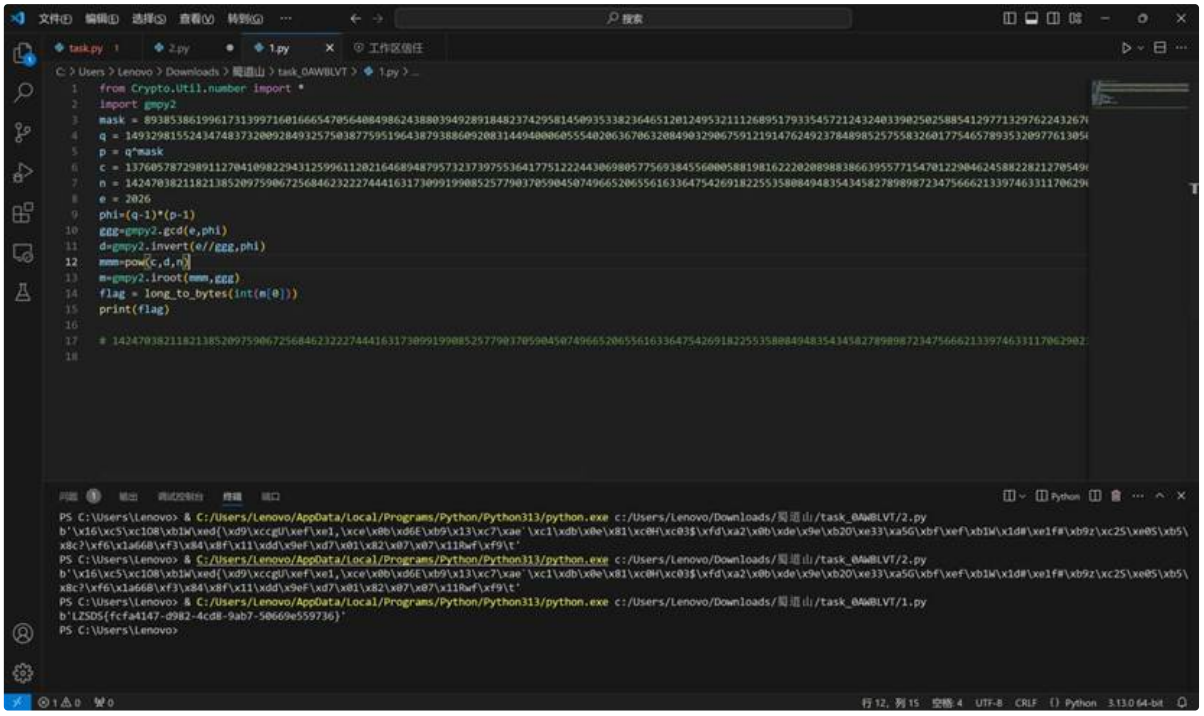
打开文件后，发现为一道rsa的题

根据代码容易得出，密文c与n，其中hint2为q，根据位运算得出p，易得n为q，p乘积与n的欧拉函数



不幸的是，发现e为偶数（悲），不能直接求逆元

那么发挥经验之力，运用典型案例，如图所示代码




```
from Crypto.Util.number import *
import gmpy2
mask =
893853861996173139971601666547056408498624388039492891848237429581450935338236
465120124953211126895179335457212432403390250258854129771329762243267072273 0
q =
149329815524347483732009284932575038775951964387938860920831449400060555402063
670632084903290675912191476249237848985257558326017754657893532097761305064 7
p = q^mask
c =
137605787298911270410982294312599611202164689487957323739755364177512224430698
057756938455600058819816222020898838663955771547012290462458822821270549691142
103071751165741784288230438170419562075032992207210421365158639796555782104995
120449177815663039476812512486455042739954026307014805905058404734127656 62
```

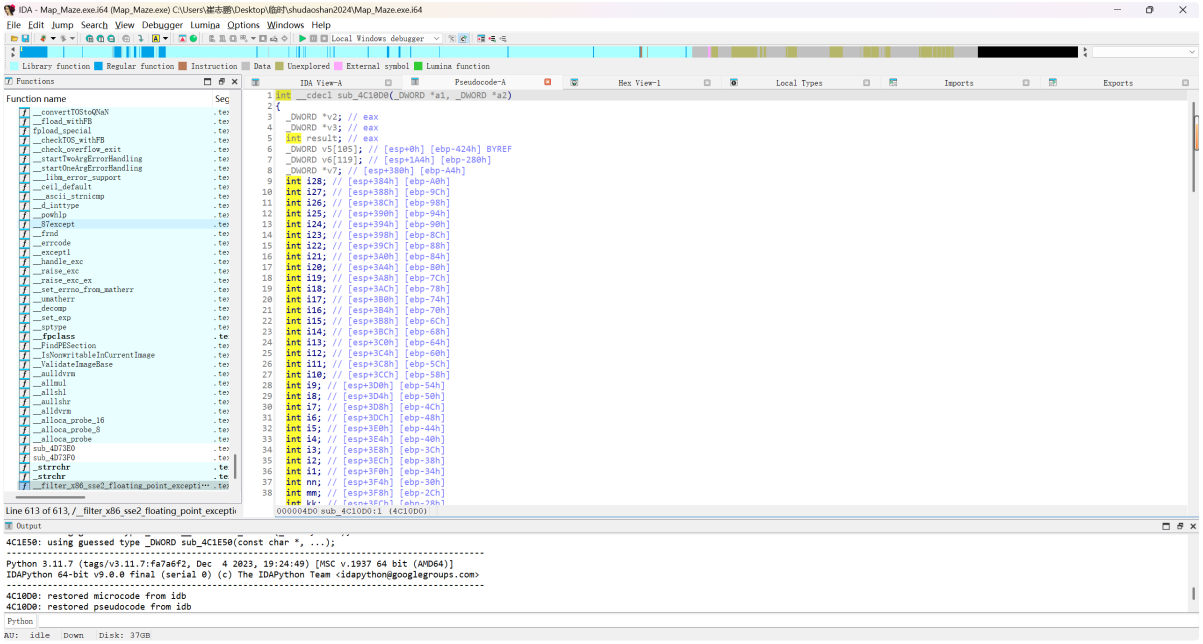
```
n =
1424703821182138520975906725684623222744416317309919908525779037059045074966520655616336475426918225535808494835434582789898723475666213397463311706290237081185546666535178402712533311266307508539567650112175978669972014909857643314181773756492877942072553979333583027422920631699946130992700052318822280165 9
e = 2026
phi=(q-1)*(p-1)
ggg=gmpy2.gcd(e,phi)
d=gmpy2.invert(e//ggg,phi)
mmm=pow(c,d,n)
m=gmpy2.iroot(mmm,ggg)
flag = long_to_bytes(int(m[0]))
print(flag)
```

得出flag

Reverse

Map_maze

IDA打开，找到构造迷宫的函数：



把代码复制一下，稍微改改便可运行：

```
#include <iostream>
#include <cstdio>
#include <cstdlib>
#include <cstring>
#include <cctype>
#include <cmath>
#include <vector>
#include <algorithm>
#include <stack>
#include <set>
#include <map>
#include <ctime>
```

```
#include <unistd.h>
#include "defs.h"
// #include <bits/stdc++.h>

using namespace std;
typedef long long LL;
typedef long double DD;

int main()
{
    int v5[225] = {0};
    int *v6 = &v5[105];
    for ( int k = 1; k < 15; ++k )
    v5[k] = 1;
  for ( int m = 9; m < 15; ++m )
    v5[m + 15] = 1;
  for ( int n = 0; n < 2; ++n )
    v5[n + 30] = 1;
  for ( int ii = 3; ii < 8; ++ii )
    v5[ii + 30] = 1;
  for ( int jj = 9; jj < 15; ++jj )
    v5[jj + 30] = 1;
  for ( int kk = 0; kk < 2; ++kk )
    v5[kk + 45] = 1;
  for ( int mm = 3; mm < 8; ++mm )
    v5[mm + 45] = 1;
  for ( int nn = 12; nn < 15; ++nn )
    v5[nn + 45] = 1;
  for ( int i1 = 0; i1 < 2; ++i1 )
    v5[i1 + 60] = 1;
  for ( int i2 = 7; i2 < 10; ++i2 )
    v5[i2 + 60] = 0;
  v5[67] = 1;
  for ( int i3 = 11; i3 < 15; ++i3 )
    v5[i3 + 60] = 1;
  for ( int i4 = 0; i4 < 2; ++i4 )
    v5[i4 + 75] = 1;
  for ( int i5 = 3; i5 < 6; ++i5 )
    v5[i5 + 75] = 1;
  for ( int i6 = 11; i6 < 15; ++i6 )
    v5[i6 + 75] = 1;
  for ( int i7 = 0; i7 < 2; ++i7 )
    v5[i7 + 90] = 1;
  v5[92] = 0;
  for ( int i8 = 3; i8 < 6; ++i8 )
    v5[i8 + 90] = 1;
  for ( int i9 = 7; i9 < 10; ++i9 )
    v5[i9 + 90] = 1;
  for ( int i10 = 11; i10 < 15; ++i10 )
    v5[i10 + 90] = 1;
  v6[0] = 1;
  v6[1] = 0;
  v6[2] = 0;
  v6[3] = 1;
```

```
  for ( int i11 = 4; i11 < 6; ++i11 )
    v6[i11] = 1;
  for ( int i12 = 7; i12 < 10; ++i12 )
    v6[i12] = 1;
  for ( int i13 = 11; i13 < 15; ++i13 )
    v6[i13] = 1;
  for ( int i14 = 0; i14 < 2; ++i14 )
    v6[i14 + 15] = 1;
  for ( int i15 = 7; i15 < 10; ++i15 )
    v6[i15 + 15] = 1;
  for ( int i16 = 11; i16 < 15; ++i16 )
    v6[i16 + 15] = 1;
  for ( int i17 = 0; i17 < 6; ++i17 )
    v6[i17 + 30] = 1;
  for ( int i18 = 7; i18 < 10; ++i18 )
    v6[i18 + 30] = 1;
  for ( int i19 = 11; i19 < 15; ++i19 )
    v6[i19 + 30] = 1;
  for ( int i20 = 0; i20 < 6; ++i20 )
    v6[i20 + 45] = 1;
  for ( int i21 = 11; i21 < 15; ++i21 )
    v6[i21 + 45] = 1;
  for ( int i22 = 0; i22 < 9; ++i22 )
    v6[i22 + 60] = 1;
  for ( int i23 = 13; i23 < 15; ++i23 )
    v6[i23 + 60] = 1;
  for ( int i24 = 0; i24 < 9; ++i24 )
    v6[i24 + 75] = 1;
  v6[84] = 0;
  v6[85] = 1;
  v6[86] = 1;
  v6[87] = 0;
  for ( int i25 = 13; i25 < 15; ++i25 )
    v6[i25 + 75] = 1;
  for ( int i26 = 0; i26 < 9; ++i26 )
    v6[i26 + 90] = 1;
  v6[99] = 0;
  v6[100] = 1;
  v6[101] = 1;
  v6[102] = 0;
  for ( int i27 = 13; i27 < 15; ++i27 )
    v6[i27 + 90] = 1;
  for ( int i28 = 0; i28 < 12; ++i28 )
    v6[i28 + 105] = 1;

    for(int i = 0; i < 15; i++)
    {
        for(int j = 0; j < 15; j++)
        {
            if(v5[i * 15 + j])
                cout << "1 ";
            else
                cout << "0 ";
        }
```

```
        cout << endl;
    }
    return 0; //-22 61 13 92
}

//Y0u_4Re_900d_47_id4
/*
input ^ 23 ^ v23
*/
```

运行后可以得到迷宫结构，分析可以发现有多条路线：

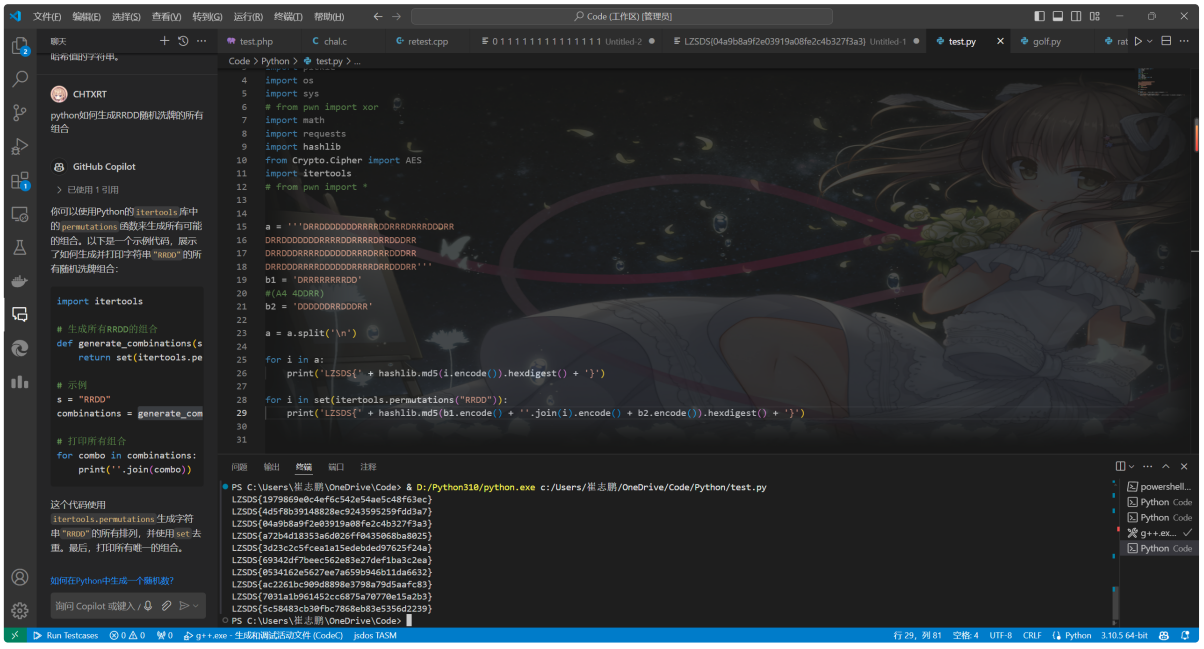
```
0 1 1 1 1 1 1 1 1 1 1 1 1 1 1
0 0 0 0 0 0 0 0 0 1 1 1 1 1 1
1 1 0 1 1 1 1 1 0 1 1 1 1 1 1
1 1 0 1 1 1 1 1 0 0 0 0 1 1 1
1 1 0 0 0 0 0 1 0 0 0 1 1 1 1
1 1 0 1 1 1 0 0 0 0 0 1 1 1 1
1 1 0 1 1 1 0 1 1 1 0 1 1 1 1
1 0 0 1 1 1 0 1 1 1 0 1 1 1 1
1 1 0 0 0 0 0 1 1 1 0 1 1 1 1
1 1 1 1 1 1 0 1 1 1 0 1 1 1 1
1 1 1 1 1 1 0 0 0 0 0 1 1 1 1
1 1 1 1 1 1 1 1 1 0 0 0 0 1 1
1 1 1 1 1 1 1 1 1 0 1 1 0 1 1
1 1 1 1 1 1 1 1 1 0 1 1 0 1 1
1 1 1 1 1 1 1 1 1 1 1 1 0 0 0

DRR
DDD RRRRRRDD
DDDDRRRR RRRRDDDD\DDRR RRDD DRDR RDRD RDDR
DDRRR\DDDDD
DR RD\D
RRDDDRR

DRRDDDDDDDRRRRDDRRRDRRRDDDRR
DRRDDDDDDDRRRRDDRRRRDRRDDDRR
DRRDDDRRRRDDDDDDRRRDRRRDDDRR
DRRDDDRRRRDDDDDDRRRRDRRDDDRR
DRRRRRRRRDD(A4 4DDRR)DDDDDDRRDDDRR
```

使用Python代码把所有情况的flag列出来：

```
from Crypto.Util.number import *
import base64
import pickle
import os
import sys
import math
import requests
import hashlib
from Crypto.Cipher import AES
import itertools
```

```python
# from pwn import *


a = '''DRRDDDDDDDRRRRDDRRRDRRRDDDRR
DRRDDDDDDDRRRRDDRRRDRRDDDRR
DRRDDDRRRRDDDDDDRRRDRRRDDDRR
DRRDDDRRRRDDDDDDRRRRDRRDDDRR'''
b1 = 'DRRRRRRRRDD'
#(A4 4DDRR)
b2 = 'DDDDDDRRDDDRR'

a = a.split('\n')

for i in a:
    print('LZSDS{' + hashlib.md5(i.encode()).hexdigest() + '}')

for i in set(itertools.permutations("RRDD")):
    print('LZSDS{' + hashlib.md5(b1.encode() + ''.join(i).encode() +
b2.encode()).hexdigest() + '}')
```

运行得到许多flag:



尝试后发现第一个就是正确flag。